None

Efficiently Retrieving Multiple Portfolio Data with the Bloomberg API: A Python Example

How can you use the Bloomberg API without struggling with sessions, events, requests, and responses, while also achieving good performance? The answer is to use Python with the XBBG package.

If you want to automate data extraction from Bloomberg, you need to use the Bloomberg API, which allows you to interface with Bloomberg programmatically. Multiple programming languages can be used based on your preferences. Naturally, as a Windows and Office user, my first choices were VBA in Excel, MS Access, and .NET C# to implement .dll or .xll in different ways. Eventually, I discovered Python, and it surprised me. Pandas DataFrames are a natural fit for retrieving Bloomberg data, so you need less mapping code to easily obtain your results.

To execute the following code, you just need an Excel file as input. here you can download the template input file, which you need to fill with your Bloomberg Portfolios list (PTF_CODES sheet) and the list of fields (FIELDS sheet) you want to request from Bloomberg for each of the securities in each portfolio. Just download the file, input your data, and save it in your preferred location.

Before running the code, you need to install:

1. The Bloomberg Official Python API
2. the Bloomberg XBBG API which provides easier access to the Official Bloomberg API (acting as a second API layer that improves your programming experience)

If you are using the Anaconda distribution, you can install the Bloomberg Python API through ‘conda’ :

1. - CondaBloomberg API
2. then install the xbbg package as described here : - xbbg - Bloomberg API

Please note that local data usage must be compliant with the Bloomberg Datafeed Addendum (full description in DAPI): To access Bloomberg data via the API (and use that data in Microsoft Excel), your company must sign the 'Datafeed Addendum' to the Bloomberg Agreement. This legally binding contract describes the terms and conditions of your use of the data and information available via the API (the "Data"). The most

fundamental requirement regarding your use of Data is that it cannot leave the local PC you use to access the BLOOMBERG PROFESSIONAL service.

In [ ]:
```
# Set pandas display options to show all columns and up to 1000 rows
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 1000)
```

In [ ]:
```
"""
This class, GM_BLP_PTF, is designed to interact with Bloomberg's API to retrieve portfolio
and securities data, process and organize this data into DataFrames, and then export the
merged data to an Excel file. It initializes several DataFrames and lists to hold this data,
loads input data from an Excel file, retrieves relevant financial data from Bloomberg, and
provides an export function to save the processed data. The working directory refers to
the directory from which data is imported and to which data is exported.
"""

class GM_BLP_PTF:
    """
    Initialize various data structures for portfolio, securities, and other fields.
    """
    def DataFrameInit(self):
        self.PTF_CODES = pd.DataFrame()  # DataFrame to store portfolio codes
        self.BLP_FIELDS = pd.DataFrame()  # DataFrame to store Bloomberg fields
        self.DF_PORTFOLIOS = pd.DataFrame()  # DataFrame for portfolios data
        self.DF_SECURITIES = pd.DataFrame()  # DataFrame for securities data
        self.DF_PTF_ALL = pd.DataFrame()  # DataFrame to store the merged portfolios and securities data
        self.Securities = list()  # List to store unique securities
        self.Ptfcodes = list()  # List to store portfolio codes
        self.Fields = list()  # List to store fields to be retrieved
        self.Overrides = {}  # Dictionary to store override values if any

    """
    Constructor to initialize the working directory and call DataFrameInit.
    The working directory is the directory where data is imported from and exported to.
    """
    def __init__(self, WDIR=r'C:\Dati\BLP_API_DATA'):
        self.DataFrameInit()
        self._wdir = WDIR  # Set the working directory
```

```python
    # Property to get the working directory (directory for data import/export)
    @property
    def wdir(self):
        return self._wdir

    # Setter to update the working directory (directory for data import/export)
    @wdir.setter
    def wdir(self, value):
        self._wdir = value

    """
    Method to remove spaces from a given string.
    """
    def remove(self, string):
        return string.replace(" ", "")

    """
    Method to load input data from an Excel file in the working directory.
    """
    def Load_Input(self, str_file_name=r'\BLP_API_INPUT.xlsx'):
        self.InputFile = self.wdir + str_file_name  # Construct the full path to the input file

        # Load portfolio codes from the input Excel file and filter selected ones
        self.PTF_CODES = pd.read_excel(self.InputFile, sheet_name='PTF_CODES')
        self.Ptfcodes = list(self.PTF_CODES.loc[self.PTF_CODES['Selezione'] == 1]['PTF_CODE'] + " Client")

        # Load Bloomberg fields from the input Excel file
        self.BLP_FIELDS = pd.read_excel(self.InputFile, sheet_name='FIELDS')
        self.Fields = list(self.BLP_FIELDS['Fields'])

    """
    Method to get data from Bloomberg using the specified portfolio codes and fields.
    """
    def Get_BLP_Data(self):

        # Fetch portfolio data using Bloomberg API
        DF = blp.bds(self.Ptfcodes, "PORTFOLIO_DATA", True)
        DF['PTF_CODE'] = DF.index  # Add portfolio code as a column
        DF['key'] = DF['security'].map(self.remove)  # Create a key column by removing spaces from security names
        self.DF_PORTFOLIOS = DF  # Store portfolio data
```

```
        # Get unique securities and fetch their data using Bloomberg API
        self.Securities = list(DF.security.unique())
        DF_SECURITIES = blp.bdp(self.Securities, self.Fields)
        DF_SECURITIES['key'] = DF_SECURITIES.index  # Set key column
        DF_SECURITIES['key'] = DF_SECURITIES['key'].map(self.remove)  # Remove spaces from key
        self.DF_SECURITIES = DF_SECURITIES  # Store securities data

        # Merge portfolio and securities data on the key column
        self.DF_PTF_ALL = DF_SECURITIES.merge(DF, left_on='key', right_on='key')

    """
    Method to export the merged portfolio data to an Excel file in the working directory.
    """
    def Export(this, ExportDir=None):
        if ExportDir is None:
            ExportDir = this.wdir  # Use working directory if no export directory is specified

        # Export the merged data to an Excel file
        this.DF_PTF_ALL.to_excel(ExportDir + r'\DF_PORTFOLIOS.xlsx')
```

In [ ]:
```
# Import necessary libraries for numerical operations, data manipulation, and Bloomberg API interaction
import time
import numpy as np
import pandas as pd
import blpapi
from xbbg import blp # xbbg provides a simplified interface to the Bloomberg API
```

In [ ]:
```
def execute_all(WDIR, STR_INPUT_FILE, EXPORT_DIR):
  # Create an instance of the GM_BLP_PTF class, passing the working directory
  OBJ = GM_BLP_PTF(WDIR)
  # Load input data from the specified file
  OBJ.Load_Input(STR_INPUT_FILE)
  # Retrieve Bloomberg data
  OBJ.Get_BLP_Data()
  # Export the processed data to the export directory
  OBJ.Export(EXPORT_DIR)
```

In [ ]:
```
def measure_execution_time(func, *args, **kwargs):
    """
    Measures the execution time of a given function.
```

```
    Parameters:
    func (callable): The function whose execution time you want to measure.
    *args: Arguments to pass to the function.
    **kwargs: Keyword arguments to pass to the function.
    Returns:
    execution_time (float): The time it took to execute the function, in seconds.
    """
    start_time = time.time()  #we get the Start Time
    func(*args, **kwargs)     #Execute the function with the given arguments
    end_time = time.time()    #we measure the end time
    print(f"Execution time: {end_time - start_time:.2f} seconds")
    return  end_time - start_time
```

In [ ]:
```
# Define paths for working directory, export directory, and input file
WDIR = r'C:\Dati\BLP_API_DATA' # Directory for storing input and intermediate files
EXPORT_DIR = r'C:\Dati\BLP_API_DATA' # Directory for saving the exported results
STR_INPUT_FILE = r'\BLP_API_INPUT.xlsx' # Name of the Excel file containing input data
```

In [ ]:
```
ExecutionTime = measure_execution_time(execute_all,WDIR, STR_INPUT_FILE, EXPORT_DIR)
```